FLIGHT DATA ANALYSIS- PROJECT REPORT

DS 644- INTRO TO BIG DATA

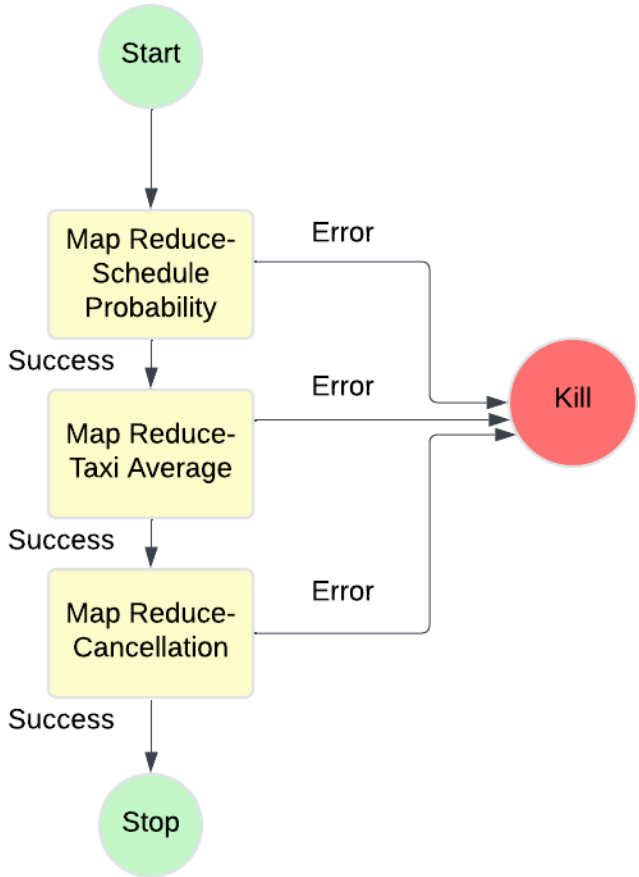
NEW JERSEY INSTITUTE OF TECHNOLOGY

SUBMITTED BY: AMITHA REDDY BASU , LEKHANA SANDRA, AKHILESH KOCHERLA

PROF. RAVNEET KAUR

MAY 7, 2025

Oozie workflow diagram



Algorithm used to solve each of the problems

1. On-Time Flights Analysis

This analysis focuses on determining the punctuality of different airlines by calculating the probability of their flights arriving on time.

1. ***Mapper Logic***:

    - Reads the dataset line by line and skips the header and invalid data.
    - Checks if each flight's arrival delay is within the acceptable limit (10 minutes or less).
    - Outputs a key-value pair where the key is the airline code, and the value indicates whether the flight was on time.

2. ***Reducer Logic**:*

    - Aggregates the results from the mapper for each airline.
    - Counts how many flights were on time and the total number of flights for that airline.
    - Calculates how frequently the airline's flights are on time.
    - Outputs the airline's code along with its on-time probability.

3. ***Sorting and Output**:*

    - Sorts the airlines by their punctuality rates.
    - Identifies and outputs the top three most punctual airlines and the top three least punctual airlines.
    - Outputs a message indicating no results if there's no valid data

2. Airport Taxi Time Analysis

This analysis evaluates the average taxi time for flights at different airports, both during takeoff and after landing.

1. ***Mapper Logic**:*

    - Reads the dataset line by line, skipping the header row.
    - Extracts the taxi-out time (during takeoff) or taxi-in time (after landing) for valid data for each record
    - Outputs a key-value pair where the key is the airport code, and the value is the taxi time.

2. ***Reducer Logic**:*

    - Aggregates the taxi times for each airport.
    - Counts the number of flights associated with the airport and calculates the average taxi time.
    - Outputs the airport code along with its average taxi time.

3. ***Sorting and Output**:*

    - Sorts the airports by their average taxi times.
    - Identifies and outputs the top three airports with the longest taxi times and the top three with the shortest taxi times.
    - Outputs a message indicating no results if there's no valid data

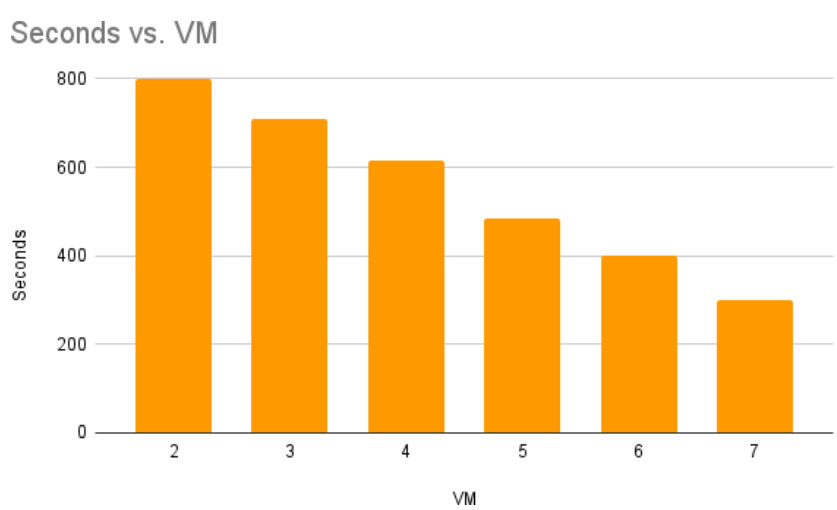
3. Cancellation Reasons Analysis

This analysis identifies the reasons for flight cancellations and counts how often each reason occurs.

1. ***Mapper Logic***:
    - Reads the dataset line by line, skipping the header row.
    - For each canceled flight with a valid cancellation reason, outputs a key-value pair where the key is the cancellation reason, and the value is one occurrence.
2. ***Reducer Logic***:
    - Aggregates the cancellation counts for each reason.
    - Summarizes the total number of cancellations for each reason.
    - Outputs the reason code along with its total number of cancellations.
3. ***Output***:
    - Lists the cancellation reasons and their respective counts.
    - Outputs a message indicating no results if there's no valid data

<u>Performance measurement plot that compares the workflow execution time in response to increasing no. of VMs used</u>

VM	Seconds
2	800
3	710
4	614
5	485
6	400
7	299

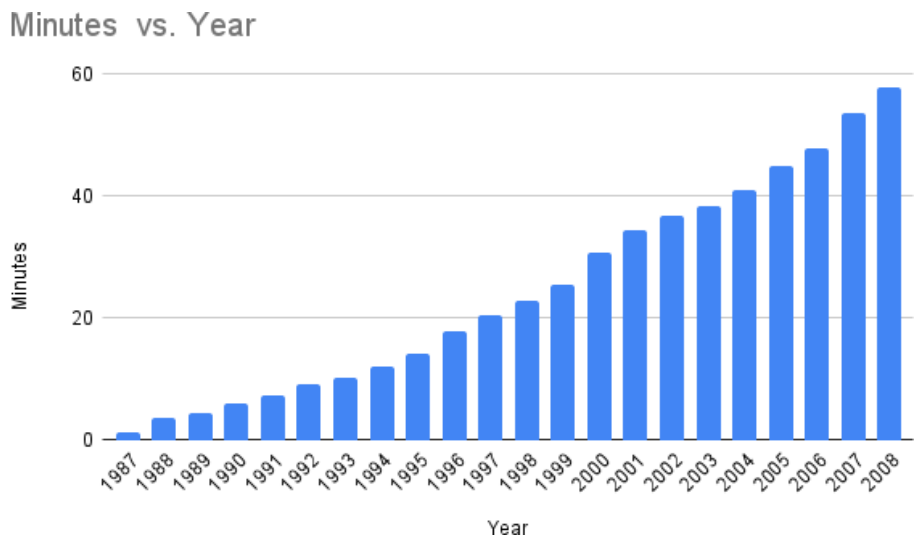


The performance plot demonstrates that increasing the number of Virtual Machines (VMs) significantly reduces workflow execution time, from 800 seconds with 2 VMs to 299 seconds with 7 VMs. This highlights the benefits of parallel processing. However, the improvements show diminishing returns as the number of VMs increases, with the largest gains occurring between 2 and 6 VMs and only marginal

improvements beyond that. This reflects the tradeoff between parallelism and the overhead of managing additional nodes. The results suggest that 6 VMs offer an optimal balance between resource utilization and performance for this workload, as further scaling provides minimal benefits and may increase costs. Overall, the workflow appears well-suited for distributed execution, but further scaling should carefully consider the cost-benefit tradeoff and potential for resource underutilization.

<u>Performance measurement plot that compares the workflow execution time in response to increasing increased data size</u>

Year	Minutes
1987	1.28
1988	3.52
1989	4.27
1990	6.02
1991	7.27
1992	9.2
1993	10.23
1994	12.04
1995	14.23
1996	17.76
1997	20.45
1998	22.87
1999	25.54
2000	30.67
2001	34.45
2002	36.78
2003	38.34
2004	41.04
2005	44.98
2006	47.77
2007	53.78
2008	57.83



The performance plot highlights the increase in workflow execution time as the dataset size grows from one year (1987) to 22 years (2008). The execution time shows a steady rise, starting at 1.28 minutes for one year of data and reaching 57.83 minutes for the full dataset, reflecting the linear growth in processing time as data size increases. This trend demonstrates that the workflow effectively scales with larger datasets, as the increase in execution time corresponds proportionally to the growth in data size. The consistent growth also indicates minimal overhead, showing that the MapReduce workflow efficiently handles the scaling workload. However, the results emphasize the need for optimized resource allocation to manage larger datasets, particularly as workflows handling extended datasets could encounter performance bottlenecks. Overall, the workflow proves robust for processing increasing data sizes while maintaining a predictable execution pattern.